

A SOLUTION FOR A GREENER FUTURE

Viktória Kubišová

TABLE OF CONTENTS

1. Problem Definition
2. Solution Overview
3. Results & Evaluation
4. Computer Vision
5. Error Analysis & Iteration
6. Robotics
7. Assumptions
8. Limitations
9. Next steps



PROBLEM DEFINITION

- The Netherlands Plant Eco-phenotyping Centre (**NPEC**)
- 2nd module: *Plant-Microbe Interaction Phenotyping*
- In-vitro root system (Hades)
 - **10,000+**
Arabidopsis thaliana
seedlings in **2,000+** Petri dishes

CLIENT OBJECTIVES

- **Automated analysis** of plant root systems and **robotic inoculation** of plants



5 plants per Petri dish, photographed every day

SOLUTION OVERVIEW

➤ Integrating **computer vision** and **robotics** into a fully automated, **end-to-end inoculation pipeline**

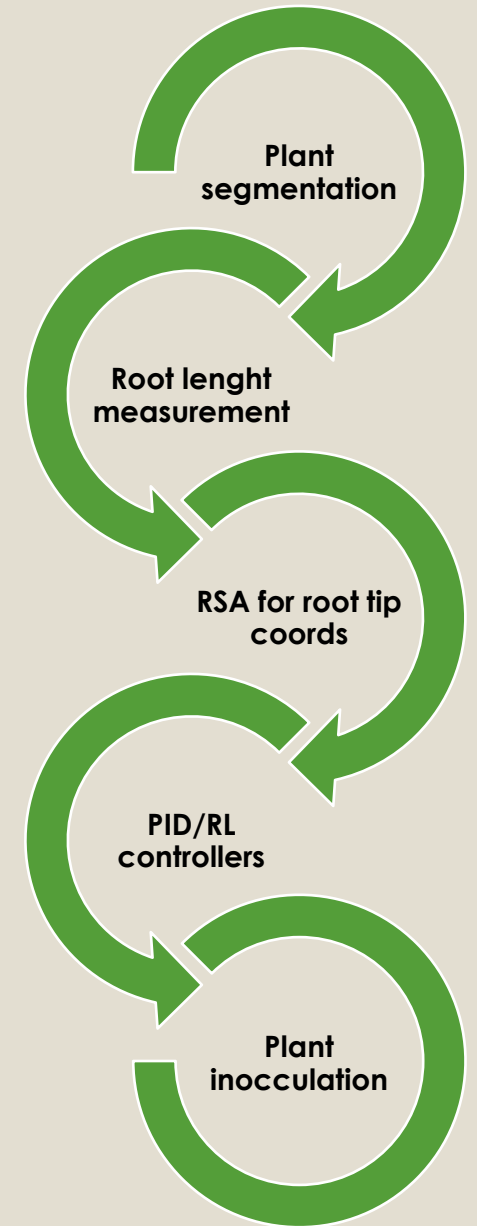
What's in it for NPEC?

High-throughput
root inoculation

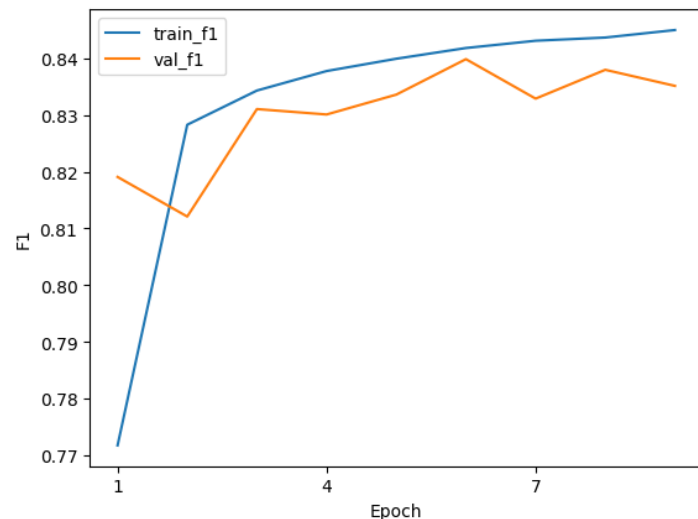
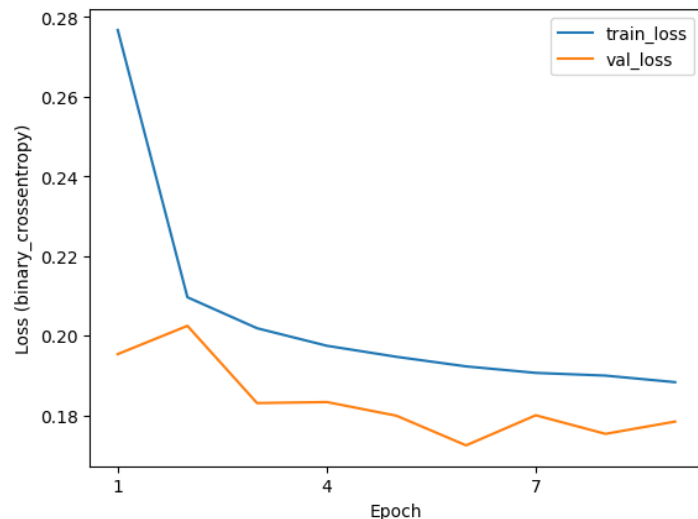
Discovery of
**stress-resistant
genotypes**

Scalability &
consistency in
plant
phenotyping

Research on
**root-microbe
interactions**



Validation F1 scores: 78-84%



RESULTS & EVALUATION

- Multiple simple **U-Net models** with extensive **hyperparameter tuning**
- Trained on **root-rich grayscale datasets** using **256×256** and **512×512** patches (step size = half the patch)
- Optimized to **minimize validation loss**
- **Key training details and Hyper-tuning:**
 - Metrics: Custom F1 score, accuracy
 - Batch Sizes: 8, 16, 32
 - Loss Functions: binary_crossentropy, bce + dice loss, focal + dice loss
 - Learning Rate Scheduling
 - Real-time augmentation using ImageDataGenerator

COMPUTER VISION - BEST ITERATION

Strategy

- **Pretrained U-Net** with **256×256 patch size**, overlapping at **step size 128**

Input Pre-processing & Inference

- **Petri dish cropping** via external contour detection
- **Padding** to match patch dimensions
- **Patchification** into overlapping 256×256 tiles
- **Model prediction** on each patch
- **Unpatch, unpad, uncrop to full mask**

Post-processing

- **Thresholding** at **0.10** to binarize predicted masks
- **Morphological closing**:
 - **Kernel size**: 3×3
 - **Dilation**: 3 iterations
 - **Erosion**: 2 iterations
- **Top & petri dish crop**: removes **15%** of image height and crops to petri dish size

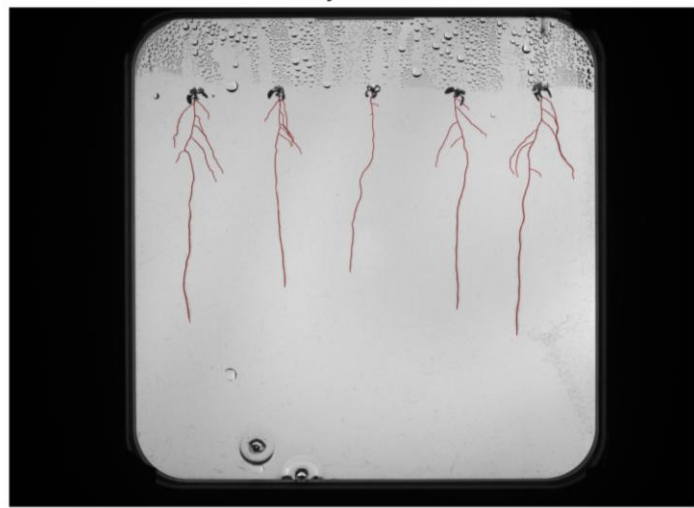
Component filtering

- **Min area**: 100 px
- **Aspect ratio** ≥ 1.5
- **Vertical start** $\leq 30\%$ of image height
- **Angle** $\geq 70^\circ$ for small roots (area < 400 px)
- **Edge margin** ≥ 250 px from sides

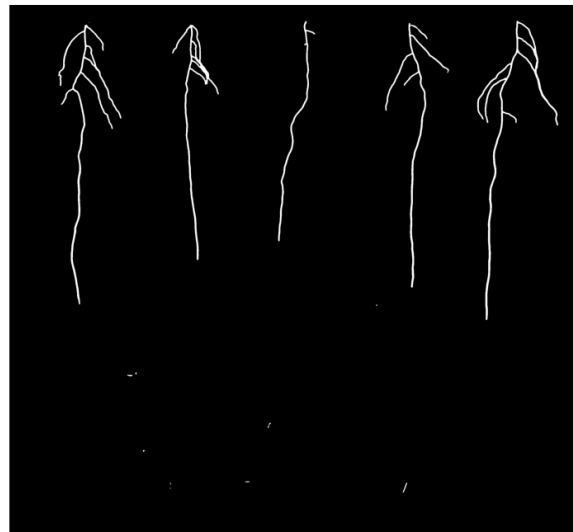
Root Segmentation & Length Estimation

- **Per-plant segmentation** into **5 vertical bands** based on **centroid position**
- **Root length estimation** via **Dijkstra's algorithm**:
 - Runs on **skeletonized root mask**
 - Finds **shortest path** from **topmost** to **bottommost** node

Overlay: Prediction



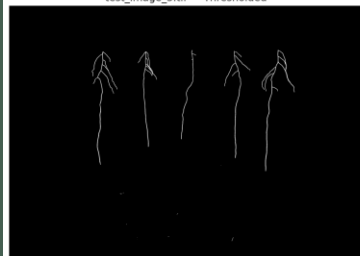
test_image_5.tif — Cropped Top + Dish Width



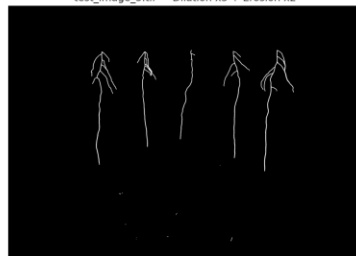
test_image_5_plant_1



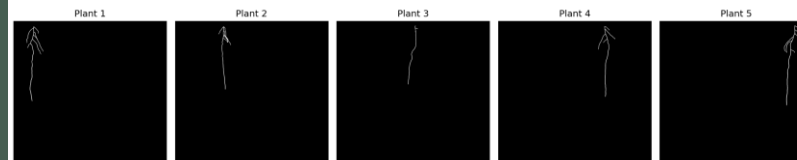
test_image_5.tif — Thresholded



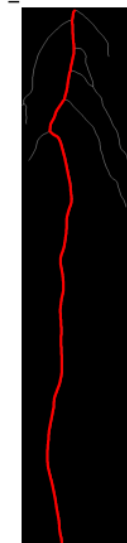
test_image_5.tif — Dilation x3 + Erosion x2



test_image_5.tif — Segmented Root Regions (w/ bbox edge filter)



test_image_5_plant_1 — Primary Length: 1440.07 px



Overview

Data

Discussion

Leaderboard

Rules

Team

Submissions

✓ 🕒 013_submission.csv Complete (after deadline) · 2d ago	11.960	7.068	☐
✓ 🕒 016_submission.csv Complete (after deadline) · 2d ago	12.219	22.598	☐
✓ 020_submission.csv Complete · 5d ago	11.870	4.586	☒
✓ 021_submission.csv Complete · 5d ago	11.917	5.795	☒
✓ 011_submission.csv Complete · 6d ago · manual closing	13.373	6.833	☐
✓ 012_submission.csv Complete · 6d ago · with the 02 model, and manual closing	12.486	7.810	☐
✓ 010_submission.csv Complete · 6d ago · lower mask thresholding, skeleton path	13.422	8.724	☐
✓ 009_submission.csv	12.049	6.800	☐

ERROR ANALYSIS AND ITERATIONS

Iteration 1: Diverse datasets

- **Clean Dataset**

- Merged Y2B_23 & Y2B_24
- Removed non-root masks + unpaired images

- **Filtered Dataset**

- Kept root-rich patches (≥ 30 pixels)
- Sampled 10% background patches

- **Augmented Datasets**

- Augmented root-rich patches: **original + flipped + lightroom enhanced**
- Added:
 - **10–15% clean** backgrounds (std < 4.5)
 - **20–25% noisy** backgrounds

Iteration 2: Root Length Estimation

- **Initial methods:** Based on **top/bottom tip nodes detection** (via skan + networkx) or **scored paths** (verticality & center)
- **Final approach:** Running **Dijkstra path mapping** from **topmost** to **bottommost pixel**
 - Maps the **true root path**
 - Gave the **cleanest & most accurate results**



ERROR ANALYSIS AND ITERATIONS

Iteration 3: Segmentation method

- **Before:**
 - Detected roots **within each band**
 - Filters: **area ≥ 250 px, aspect ≥ 1.5 , start $\leq 30\%$ height**
- **After:**
 - Ran global component detection
 - Assigned by **centroid x-position**
 - Extra checks for small roots (< 400 px):
→ **angle $\geq 70^\circ$, ≥ 250 px from edges**

Iteration 4: Loss Function Update

- Switched from binary_crossentropy → **BCE + Dice Loss**
 - **Binary Crossentropy** handles pixel-wise accuracy
 - **Dice Loss** boosts overlap between predicted & true masks
- Combined to balance **class imbalance** and improve **small root detection**

```
# Define a new loss function
# Dice Loss
def dice_loss(y_true, y_pred):
    smooth = 1e-6
    y_true_f = K.flatten(y_true)
    y_pred_f = K.flatten(y_pred)
    intersection = K.sum(y_true_f * y_pred_f)
    return (1 - (2. * intersection + smooth) /
            (K.sum(y_true_f) + K.sum(y_pred_f) + smooth))

# Combo Loss: Binary Crossentropy + Dice
def bce_dice_loss(y_true, y_pred):
    bce = tf.keras.losses.binary_crossentropy(y_true, y_pred)
    return bce + dice_loss(y_true, y_pred)
```

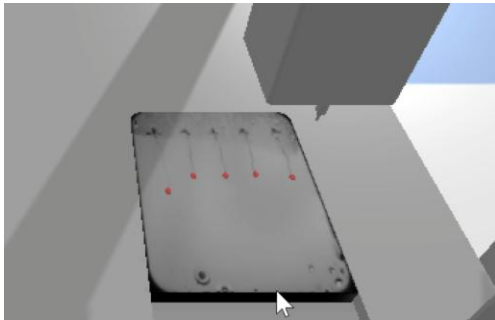
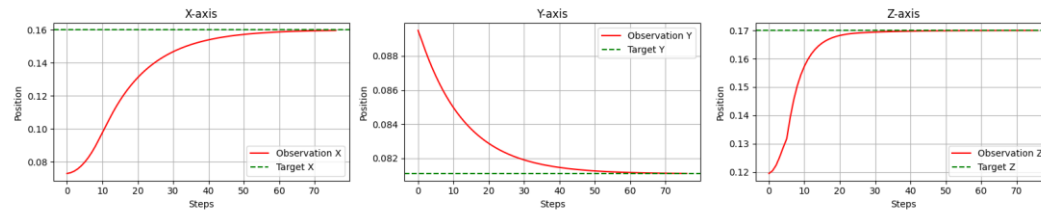
ROBOTICS

- Integrated **computer vision pipeline** with a **3D PID controller**
- The controller adjusts **X, Y, and Z** positions of the pipette in real-time

RESULTS:

- **100% accuracy within 1 mm** on all 5 roots
- **X/Y axes:** smooth convergence, **no major overshoot**
- **Z-axis:** some early jitter, but stabilized
- Average steps to reach target: **62.00** → good speed–precision balance
- **Accurate droplet placement** at root tips

Root 0 - PID Trajectory Tracking



```
[STATS] Average steps to reach target: 62.00
[STATS] Accuracy (within 1.0mm): 100.0%

[STATS] Final distances per root:
Root 0: 0.000360 m
Root 1: 0.000350 m
Root 2: 0.000360 m
Root 3: 0.000360 m
Root 4: 0.000360 m
```

test_image_7.tif — Thresholded at 0.1



ASSUMPTIONS

- **Top 15%** of image height contains noise
- Minimum root area = **100 px**
- Roots must be **vertically elongated**
 - Aspect ratio ≥ 1.5
- Small components (*area* < 400 px) must satisfy:
 - Angle $\geq 70^\circ$
 - ≥ 250 px margin from image side
- Valid roots start in the **top 30%** of image height
- 5 vertical bands, keeping only the **largest valid component** per band
- The primary root → the shortest path from the **topmost to bottommost** node

LIMITATIONS

Overlapping and dense roots make it hard for the model to identify the main root path accurately.

When **lateral root starts above** the actual primary root → the algorithm mistakenly uses this lateral root's top node as the start of the path.

When **lateral root grows below** the true primary root → the path is extended to this deeper tip, overestimating length.



NEXT STEPS

1

Train **seed and shoot segmentation models** to better find where the main root starts

2

Filter false root paths by adding logic to ignore misleading lateral roots above or below the primary root

3

Tune PID using methods like Ziegler–Nichols, Cohen–Coon, or feedforward compensation

4

Train an **RL model** to compare with the **PID controller**



THANK YOU!

Any questions?